Universidad de La Sabana
Facultad de Ingeniería

Felix Mohr & Sergio Amortegui
2026.1

Fundamentos de Inteligencia Artificial

- Sheet 8-

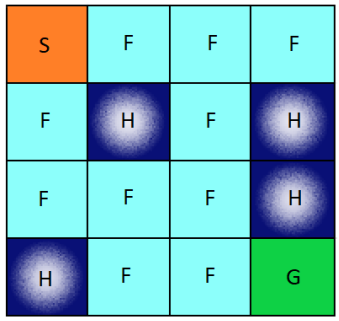
Environment & Rules. Use Python with only `numpy`. Do not use global random state; all randomness must come from a provided `numpy.random.Generator` named `rng`. Unless explicitly stated otherwise, functions that sample must accept a seed or `rng`. Keep the function signatures from the provided `.py` templates unchanged, as they will be tested automatically on Gradescope. You will only see public test outputs.

Files to implement. `solution.py` Do not rename files or functions.

Environment Considerations: Lake MDP (reuse). We consider the *Lake* environment on an $m \times n$ grid. Each cell is one of

`S` (unique start), `F` (free), `H` (hole, terminal), `G` (goal, terminal).

At each non-terminal step the agent chooses an action in $\{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}\}$. With probability 0.8 the intended move is executed; with probability 0.1 each, the move slips to the two lateral directions if the state is not an absorbing one. Bumping against borders keeps the agent in place.
Rewards depend *only* on the state *entered*: a constant step reward in `F` (-0.1), a fixed reward in `H` (-1.0), and a fixed reward in `G` ($+1.0$). Upon entering any terminal cell, the process transitions to a synthetic absorbing state $s_\perp$ with only action $\perp$ and reward 0 thereafter.



We assume throughout that the discount factor satisfies $\gamma \in (0, 1]$.

Exercise 1 (Tabular Policy – 1 Point) Implement the class `TabularPolicy`. The class should accept a dictionary mapping states to actions, where each key corresponds to a state and each value specifies the chosen action in that state.

If no dictionary is provided, the policy must behave as a uniform random policy over all admissible actions.

Exercise 2 (Policy Evaluation and Value Functions – 1 Point) Implement the following functions related to policy evaluation and value functions:

1. **Policy Evaluation.** Implement the function `policy_evaluation` that computes the state-value function $v^\pi(s)$ for all $s \in S$, given a fixed policy π. Formally, for each state $s \in S$, compute
$$v^\pi(s) \;=\; r(s) \;+\; \gamma \sum_{s'} P(s' \mid s, \pi(s))\, v^\pi(s').$$

2. **State-Action Values from State Values.** Implement the function `q_from_v` that computes the state-action value function $q^\pi(s, a)$ from a given state-value function $v^\pi(s)$. Formally, for each $(s, a) \in S \times A$, compute
$$q^\pi(s, a) \;=\; r(s) \;+\; \gamma \sum_{s'} P(s' \mid s, a)\, v^\pi(s').$$

3. **State Values from State-Action Values.** Implement the function `v_from_q` that computes the state-value function $v^\pi(s)$ from a given state-action value function $q^\pi(s, a)$ and a policy π. Formally, for each state $s \in S$,
$$v^\pi(s) \;=\; q^\pi(s, \pi(s)).$$

Exercise 3 (Policy Improvement with Advantage – 3 Points) Implement the function `policy_improvement`, which takes as input the state-value function $v^\pi(s)$ of a given policy π and returns an improved policy π'.

1. For each state $s \in S$ and action $a \in A(s)$, compute the advantage
$$A^\pi(s, a) \;=\; q^\pi(s, a) - v^\pi(s).$$

2. Update the policy according to the rule
$$\pi'(s) \;=\; \arg\max_a A^\pi(s, a).$$

3. Return the tuple (π^*, `advantage_sequence`), where π^* is the improved policy and `advantage_sequence` records the advantage values computed across iterations.

Exercise 4 (Policy Iteration – 3 Points) Implement the function `policy_iteration` for an MDP $\langle S, A, \mathbf{P} \rangle$ with an initial policy π. The function should alternate between policy evaluation and policy improvement until convergence, i.e.,
$$\pi^* = \pi.$$

The function must return the tuple (π^*, v^{π^*}), where π^* denotes the optimal policy and v^{π^*} the corresponding state-value function.

Exercise 5 (Runner with Policy Evaluation – 2 Points) Implement the function `get_optimal_policy`, which performs the following steps:

1. Construct a random `TabularPolicy` π (i.e., without a user-provided dictionary).
2. Use *policy iteration* to compute an optimal policy π^*.
3. Return the optimal policy π^*.

Exercise 6 (Practice: Additional Problems) These problems are not graded on Gradescope but will help you test and deepen your understanding.

1. **Q-values.** For a fixed policy π, show the relationship between $v^\pi(s)$ and $q^\pi(s, a)$. Verify numerically on the Lake MDP that $v^\pi(s) = q^\pi(s, \pi(s))$ for all non-terminal states.

2. **Policy Improvement Theorem.** Prove that if $q^\pi(s, \pi'(s)) \geq v^\pi(s)$ for all s, then $v^{\pi'}(s) \geq v^\pi(s)$ for all s. Check this numerically by constructing a policy π, evaluating it, improving it, and verifying the inequality on the Lake MDP.

3. **Stochastic Transitions.** In the Lake MDP, each intended action succeeds with probability 0.8 and slips with probability 0.1 each. Write explicitly the transition probabilities $P(s'|s, a)$ for the following three cases:

   - taking action UP from a non-border cell,
   - taking action LEFT from a left-border cell,
   - taking action DOWN from a cell next to a hole.

   Discuss how these probabilities affect the policy evaluation equation.

4. **Bellman Optimality.** Derive the Bellman optimality equation for the Lake MDP and explain why solving this system yields v^*. Implement an iterative solver for the optimal value function v^* and compare it to the output of your policy iteration implementation.

5. **Finite Policy Space.** Argue why there are at most $|A|^{|S|}$ deterministic policies. For a 3×3 Lake grid with 1 start, 1 goal, and 2 holes, compute explicitly how many possible policies exist.